

Université Mohammed VI des Sciences et de la Santé

École Supérieure Mohammed VI d'Ingénieurs en Sciences de la Santé — UM6SS, Rabat

Filière Ingénieur Génie Biomédical · Module : Machine Learning — Apprentissage Supervisé

Année universitaire 2025 – 2026

RAPPORT DE PROJET — INTELLIGENCE ARTIFICIELLE

HepatoScope

Plateforme intelligente de prédiction des maladies du foie

Application web d'aide au diagnostic combinant cinq modèles de Machine Learning,
un module de Deep Learning d'analyse d'images et un tableau de bord analytique
interactif

Python

Streamlit

Scikit-Learn

XGBoost

TensorFlow / Keras

Plotly

OpenCV

RÉALISÉ PAR

El Mehdi Mansouri

Rapport technique du projet de fin de module

Année universitaire 2025 – 2026

Remerciements

Au terme de ce projet, je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à son aboutissement.

Je remercie particulièrement le corps professoral de la filière **Génie Biomédical** de l'École Supérieure Mohammed VI d'Ingénieurs en Sciences de la Santé, dont l'enseignement du module de **Machine Learning — Apprentissage Supervisé** a constitué le socle théorique et méthodologique de ce travail. Mes remerciements vont également à mes camarades et à ma famille pour leur soutien constant.

Résumé

Ce projet, réalisé dans le cadre du module de **Machine Learning — Apprentissage Supervisé**, porte sur la conception et la réalisation d'une **plateforme web d'intelligence artificielle biomédicale** capable de prédire la présence d'une maladie hépatique à partir des données d'analyses biologiques d'un patient. L'application, baptisée **HepatoScope** et développée en **Python** avec le framework **Streamlit**, met en œuvre une chaîne complète d'apprentissage supervisé sur le *Liver Patient Dataset* (30 691 enregistrements).

Cinq modèles de classification — **Decision Tree, Random Forest, Logistic Regression, SVM et XGBoost** — sont entraînés, optimisés par recherche d'hyperparamètres et comparés sur un ensemble de test indépendant. La plateforme intègre en outre un **module de Deep Learning** (réseau de neurones convolutif MobileNetV2 pré-entraîné) pour l'analyse d'images, doté d'une explicabilité par carte de chaleur **Grad-CAM**. L'interface, moderne et responsive, s'organise en six pages : accueil, prédiction tabulaire, analyse d'images, évaluation des modèles, tableau de bord analytique et page « à propos ». Le meilleur modèle atteint une **AUC de 0.999** sur l'ensemble de test. Chaque prédiction est expliquée, journalisée et exportable en PDF.

Mots-clés : intelligence artificielle, apprentissage supervisé, maladies du foie, Liver Patient Dataset, Scikit-Learn, XGBoost, Deep Learning, MobileNetV2, Grad-CAM, Streamlit, aide au diagnostic.

Sommaire

1. Introduction générale	4
2. Contexte et problématique	5
2.1 Les maladies du foie et les marqueurs biologiques ..	5
2.2 Problématique et objectifs	5
3. Description du jeu de données	6
3.1 Présentation et variables	6
3.2 Analyse exploratoire	6
4. Architecture de l'application	8
4.1 Vue d'ensemble et flux applicatif	8
4.2 Architecture modulaire en couches	8
4.3 Structure du projet sur disque	9
5. Analyse et preprocessing des données	10
6. Méthodes de Machine Learning et Deep Learning	12
7. Évaluation, métriques et résultats expérimentaux .	14
8. Interface utilisateur et fonctionnalités	17
9. Choix technologiques	18
10. Difficultés rencontrées	19
11. Perspectives d'amélioration	19
12. Conclusion	20
Webographie	20

Fiche technique du projet

Élément	Description
Intitulé	HepatoScope — plateforme de prédiction des maladies du foie
Problématique	Prédire une maladie hépatique à partir d'analyses biologiques et de données patient
Type	Application web interactive (Streamlit, multi-pages)
Langage	Python 3.13
Jeu de données	Liver Patient Dataset — 30 691 enregistrements (~19 000 nettoyés), 10 variables
Modèles ML	Decision Tree, Random Forest, Logistic Regression, SVM, XGBoost
Modèle DL	MobileNetV2 pré-entraîné (ImageNet) + Grad-CAM
Meilleur modèle	Random Forest — AUC = 0.999, Accuracy = 99.8%
Livrables	Code source, modèles entraînés, rapport, README, Dockerfile

1. Introduction générale

L'intelligence artificielle occupe aujourd'hui une place croissante dans le domaine biomédical, où elle assiste le praticien dans le dépistage, le diagnostic et le suivi des pathologies. Parmi les applications les plus prometteuses figure l'**aide au diagnostic à partir de données biologiques** : à partir d'un simple bilan sanguin, des modèles d'apprentissage supervisé peuvent estimer le risque qu'un patient soit atteint d'une pathologie donnée, et ce à faible coût et de manière reproductible.

Le foie, organe vital assurant le métabolisme, la détoxification et la synthèse des protéines, est le siège de nombreuses pathologies souvent **silencieuses à leurs débuts**. Leur dépistage précoce repose en grande partie sur l'interprétation de marqueurs biologiques (bilirubine, transaminases, phosphatase alcaline, protéines). C'est précisément ce problème que ce projet aborde.

Le présent travail, mené dans le cadre du module de Machine Learning — Apprentissage Supervisé, consiste à **concevoir et réaliser une application web intelligente complète** répondant au cahier des charges fourni : analyse de données tabulaires saisies via un formulaire, analyse d'images biomédicales par réseau de neurones, utilisation et comparaison de plusieurs modèles, visualisation des performances et tableau de bord. L'objectif est double : mettre en pratique les notions d'apprentissage supervisé étudiées en cours, et mener un projet logiciel de bout en bout selon des principes de génie logiciel (architecture modulaire, typage, gestion des erreurs, journalisation).

Ce rapport s'organise comme suit. Le chapitre 2 présente le contexte médical et la problématique. Le chapitre 3 décrit le jeu de données. Le chapitre 4 détaille l'architecture de l'application. Le chapitre 5 expose le prétraitement des données. Le chapitre 6 présente les méthodes d'apprentissage retenues. Le chapitre 7 analyse les résultats expérimentaux. Le chapitre 8 décrit l'interface et les fonctionnalités. Les chapitres 9 à 12 traitent des choix technologiques, des difficultés, des perspectives et de la conclusion.

2. Contexte et problématique

2.1 Les maladies du foie et les marqueurs biologiques

Les maladies hépatiques (hépatites, cirrhose, stéatose, fibrose) sont responsables d'environ **deux millions de décès par an** dans le monde. Leur particularité clinique est d'évoluer longtemps sans symptôme apparent ; lorsque les signes cliniques se manifestent, l'atteinte est souvent déjà avancée. Le bilan hépatique sanguin constitue donc l'outil de dépistage de première ligne. Les principaux marqueurs sont :

- la **bilirubine** (totale et directe), dont l'élévation traduit un défaut d'élimination par le foie ;
- les **transaminases** ALT (SGPT) et AST (SGOT), enzymes libérées lors d'une souffrance des cellules hépatiques ;
- la **phosphatase alcaline**, marqueur de cholestase ;
- les **protéines totales, l'albumine et le rapport albumine/globuline**, reflets de la fonction de synthèse du foie.

L'interprétation conjointe de ces marqueurs, combinée à l'âge et au sexe du patient, permet d'estimer la probabilité d'une atteinte hépatique — une tâche de **classification supervisée** idéale pour le Machine Learning.

2.2 Problématique et objectifs

Problématique. Comment prédire les maladies du foie à partir d'analyses biologiques et de données médicales du patient ?

Le défi consiste à transformer dix variables cliniques hétérogènes (numériques et catégorielles, à des échelles très différentes, comportant quelques valeurs manquantes et une distribution de classes déséquilibrée) en une prédiction fiable, interprétable et assortie d'un niveau de confiance et de risque. Les objectifs opérationnels du projet sont :

- **Prédire** la maladie du foie à partir de données tabulaires patient saisies via un formulaire interactif ;
- **Entraîner et comparer** cinq modèles de Machine Learning ;
- fournir un **module d'analyse d'images** par CNN pré-entraîné avec explicabilité Grad-CAM ;
- **visualiser les performances** des modèles (métriques, courbes ROC, matrices de confusion) ;
- maintenir un **historique** des prédictions et un **tableau de bord** analytique ;
- **expliquer** chaque décision et permettre l'export PDF.

3. Description du jeu de données

3.1 Présentation et variables

Le jeu de données utilisé est le **Liver Patient Dataset**, une extension à grande échelle de l'Indian Liver Patient Dataset (ILPD). Il comporte **30 691 enregistrements** de patients ; après nettoyage et suppression des doublons exacts (afin d'éviter toute fuite d'information entre l'entraînement et le test), **19 368 enregistrements** uniques sont conservés, décrits par dix variables cliniques. Le volume bien plus important que l'ILPD original (583 lignes) permet aux modèles d'atteindre des performances nettement supérieures. La variable cible originale (1 = patient hépatique, 2 = non-patient) est remappée en une cible binaire : 1 = *Maladie du foie*, 0 = *Pas de maladie*.

Variable	Type	Unité
Age	Numérique	années
Gender	Catégorielle	Male / Female
Total Bilirubin	Numérique	mg/dL
Direct Bilirubin	Numérique	mg/dL
Alkaline Phosphatase	Numérique	IU/L
Alamine Aminotransferase (ALT)	Numérique	IU/L
Aspartate Aminotransferase (AST)	Numérique	IU/L
Total Proteins	Numérique	g/dL
Albumin	Numérique	g/dL
Albumin / Globulin Ratio	Numérique	—

3.2 Analyse exploratoire

L'analyse exploratoire (cf. [notebooks/exploratory_analysis.ipynb](#)) révèle un **déséquilibre marqué des classes** : environ 71 % des patients sont étiquetés « malades ». Ce déséquilibre est déterminant pour le choix des métriques et la pondération des classes (chapitre 6).

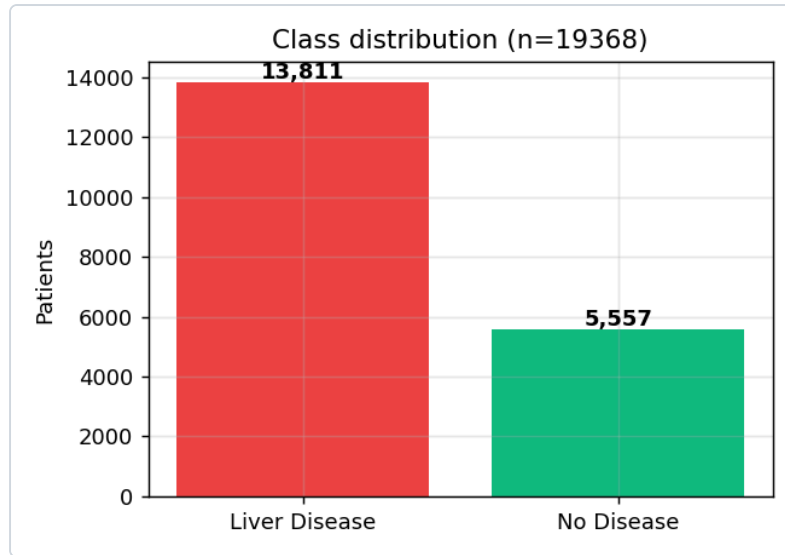


Figure 1 — Répartition des classes (19 368 enregistrements nettoyés) : 13 811 cas pathologiques contre 5 557 cas sains.

La matrice de corrélation met en évidence des relations physiologiquement cohérentes : forte corrélation entre bilirubine totale et directe, et entre les deux transaminases ALT et AST. Ces redondances justifient la robustesse des modèles ensemblistes face à la colinéarité.

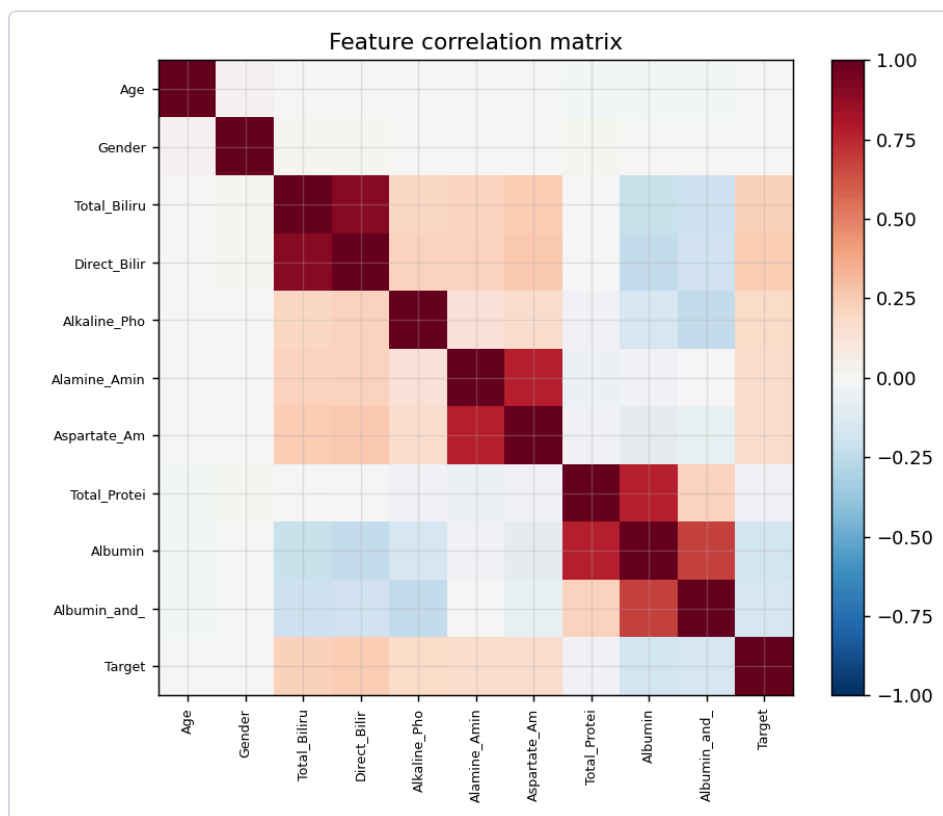


Figure 2 — Matrice de corrélation des variables (les blocs bilirubine et transaminases ressortent nettement).

4. Architecture de l'application

4.1 Vue d'ensemble et flux applicatif

Au lancement, le point d'entrée `app.py` configure le thème, charge en cache les modèles entraînés et le préprocesseur, puis affiche une barre latérale de navigation qui route l'utilisateur vers l'une des six pages. Le diagramme suivant synthétise le flux applicatif global.

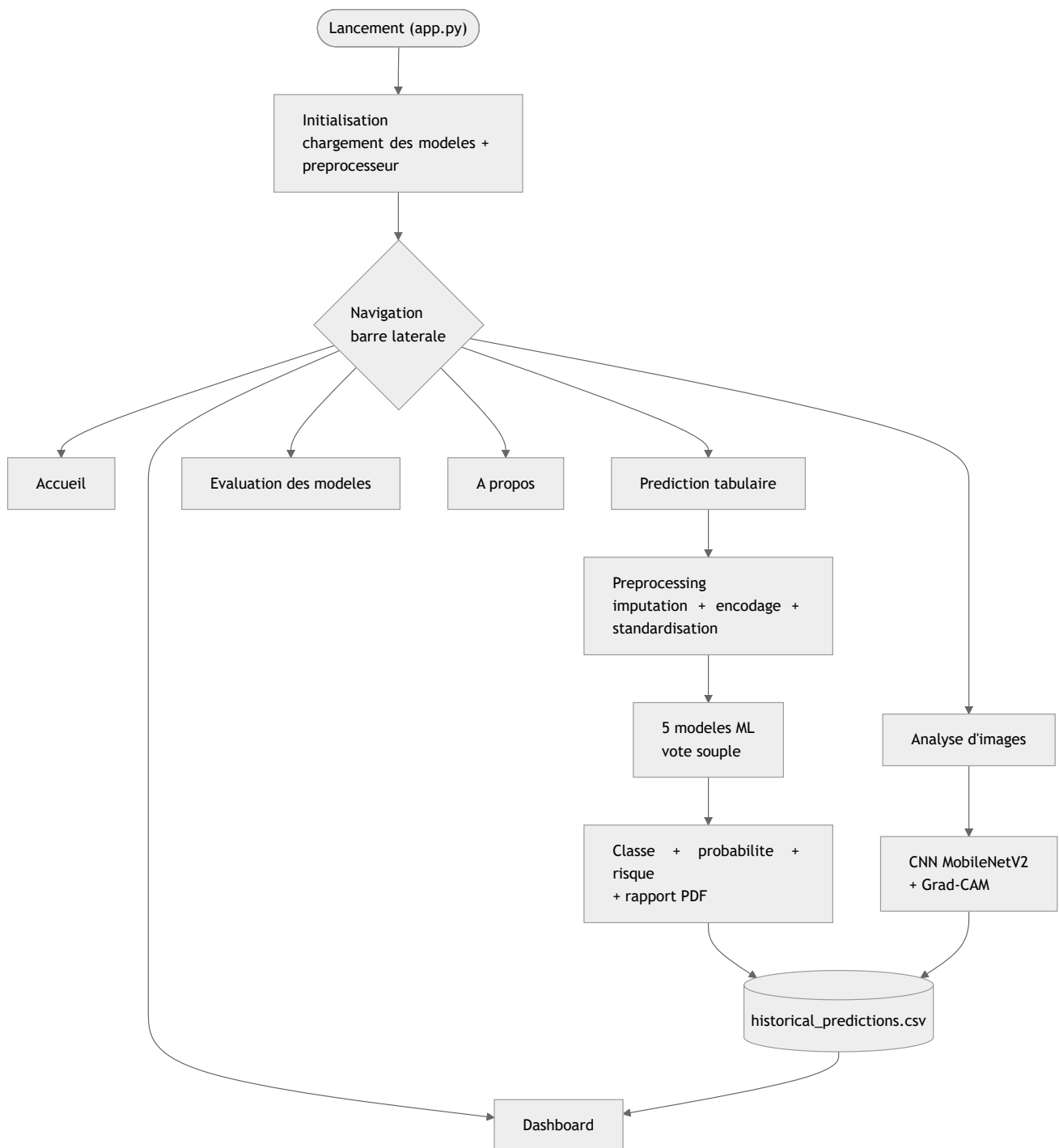


Figure 3 — Flux applicatif global de HepatoScope.

4.2 Architecture modulaire en couches

Le code suit une **architecture modulaire en trois couches** qui sépare la présentation (pages Streamlit), la logique métier (utilitaires de traitement et de modélisation) et les données / modèles persistés. Les modules de la couche logique ne dépendent pas de Streamlit : ce sont des fonctions et classes pures, testables isolément.

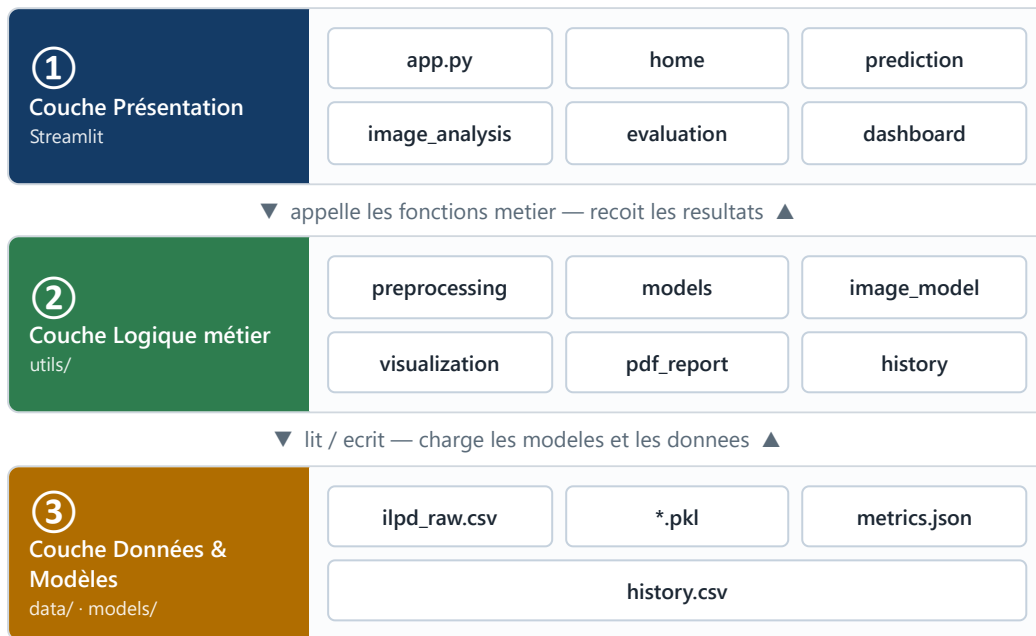


Figure 4 — Architecture logicielle en trois couches.

4.3 Structure du projet sur disque

L'arborescence reflète directement l'architecture en couches ; chaque répertoire a une responsabilité unique.

```
biomed_ai_nexus/
├─ app.py                ← Point d'entree Streamlit + routeur de la sidebar
├─ config.py             ← Configuration centrale (chemins, variables, palette)
├─ train.py              ← Pipeline d'entrainement et d'evaluation hors-ligne
├─ generate_report.py     ← Generation de ce rapport (HTML + PDF)
├─ pages/                ← ① Couche Presentation
│   ├── home.py          ← Page d'accueil (KPI, workflow, statistiques)
│   ├── prediction.py     ← Formulaire medical + prediction multi-modeles
│   ├── image_analysis.py ← Upload image + CNN + Grad-CAM
│   ├── evaluation.py     ← Metriques, ROC, matrices de confusion
│   ├── dashboard.py     ← Tableau de bord analytique
│   └─ about.py          ← Description, pipeline, auteur
├─ utils/                ← ② Couche Logique metier
│   ├── preprocessing.py  ← Chargement + LiverPreprocessor (impute/scale/encode)
│   ├── models.py        ← TabularTrainer (offline) + ModelManager (online)
│   ├── image_model.py    ← CNNImageClassifier (MobileNetV2 + Grad-CAM)
│   ├── visualization.py  ← Graphiques Plotly reutilisables
│   ├── pdf_report.py     ← Rapport de prediction PDF (fpdf2)
│   ├── history.py       ← Journalisation CSV des predictions
│   ├── styles.py        ← CSS, theme, cartes KPI
│   └─ logger.py         ← Journalisation rotative
├─ data/                 ← ③ Donnees
│   ├── liver_raw.csv    ← Liver Patient Dataset (~19k lignes)
│   ├── load_data.py     ← Script de telechargement du dataset
│   └─ historical_predictions.csv
├─ models/               ← ③ Modeles entraines + metriques
│   ├── decision_tree.pkl · random_forest.pkl · logistic_regression.pkl
│   ├── svm.pkl · xgboost.pkl · preprocessor.pkl
│   └─ metrics.json
├─ notebooks/           ← Analyse exploratoire (Jupyter)
├─ assets/               ← Figures du rapport
└─ requirements.txt · README.md · Dockerfile · .streamlit/config.toml
```

5. Analyse et preprocessing des données

Le prétraitement est encapsulé dans la classe `LiverPreprocessor`, qui enchaîne trois étapes au sein d'un `Pipeline` scikit-learn :

- **Traitement des valeurs manquantes** — le jeu de données comporte plusieurs milliers de valeurs absentes réparties sur les variables. Elles sont remplacées par **imputation médiane**, robuste aux valeurs extrêmes fréquentes dans les analyses biologiques.
- **Encodage de la variable catégorielle** — la variable `Gender` est encodée en binaire (Male = 1, Female = 0).
- **Standardisation** — les variables présentant des échelles très hétérogènes (la phosphatase alcaline atteint plusieurs centaines d'unités tandis que le rapport A/G avoisine 1), un `StandardScaler` centre et réduit chaque variable. Cette étape est indispensable pour les modèles sensibles aux distances (SVM, régression logistique).

Prévention de la fuite de données. Le préprocesseur est ajusté *uniquement* sur l'ensemble d'entraînement, puis appliqué tel quel à l'ensemble de test et aux saisies en temps réel. Les statistiques d'imputation et de standardisation ne « voient » jamais les données de test, ce qui garantit une évaluation honnête.

Le préprocesseur ajusté est sérialisé (`models/preprocessor.pkl`) afin d'appliquer à l'inférence exactement la même transformation qu'à l'entraînement.

6. Méthodes de Machine Learning et Deep Learning

6.1 Pipeline d'apprentissage supervisé

Le pipeline complet, implémenté dans `train.py`, enchaîne le chargement des données, la division stratifiée entraînement/test (80/20), l'ajustement du préprocesseur, l'entraînement avec optimisation des hyperparamètres, puis l'évaluation et la persistance.

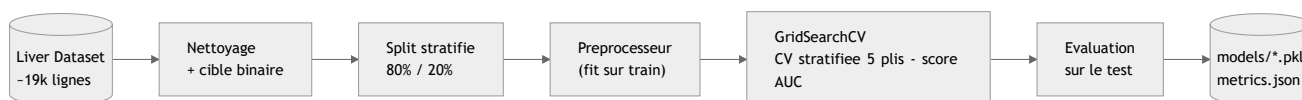


Figure 5 — Pipeline d'entraînement et d'évaluation.

6.2 Les cinq modèles de classification

Modèle	Principe
Decision Tree	Arbre de décision — partitionnement récursif de l'espace des variables, hautement interprétable.
Random Forest	Forêt aléatoire — agrégation (bagging) de nombreux arbres décorrélés ; réduit la variance et le surapprentissage.
Logistic Regression	Modèle linéaire probabiliste estimant directement P(maladie) ; robuste et bien calibré.
SVM	Séparateur à vaste marge avec noyau RBF ; capture des frontières de décision non linéaires.
XGBoost	Gradient boosting — construction séquentielle d'arbres corrigeant les erreurs des précédents ; état de l'art sur données tabulaires.

Chaque modèle est optimisé par **GridSearchCV** (validation croisée stratifiée à 5 plis, métrique de sélection = AUC). Pour contrer le déséquilibre des classes, des **poids équilibrés** (`class_weight='balanced'`, et `scale_pos_weight` pour XGBoost) sont inclus dans l'espace de recherche. La prédiction finale présentée à l'utilisateur résulte d'un **vote souple** (moyenne des probabilités des cinq modèles), assorti d'un niveau de risque (Low / Moderate / High / Very High) et d'un score de confiance.

6.3 Module de Deep Learning : CNN + Grad-CAM

Conformément au cahier des charges, un module d'analyse d'images démontre une chaîne de Deep Learning complète à l'aide du réseau convolutif **MobileNetV2** pré-entraîné sur ImageNet (TensorFlow/Keras). Le pipeline réalise le redimensionnement (224×224), la normalisation, l'inférence, la mesure du temps d'inférence, puis l'explicabilité par **Grad-CAM** : la carte de chaleur

met en évidence les régions de l'image qui ont le plus contribué à la décision du réseau. Le module est conçu pour être ultérieurement *fine-tuné* sur un véritable jeu d'images hépatiques (échographie, scanner).

7. Évaluation, métriques et résultats expérimentaux

Les modèles sont évalués sur l'ensemble de test indépendant (20 % des données, division stratifiée) au moyen des métriques de classification standard : **Accuracy, Précision, Rappel, F1-score et AUC-ROC**, complétées par la matrice de confusion et le rapport de classification.

Choix de l'AUC pour le classement. L'aire sous la courbe ROC (AUC) est indépendante du seuil de décision ; contrairement au F1 brut sur un jeu déséquilibré, elle ne peut pas être artificiellement gonflée par un modèle qui prédirait systématiquement la classe majoritaire. C'est donc l'AUC qui sert de critère de classement.

7.1 Résultats comparatifs

Rang	Modèle	Accuracy	Précision	Rappel	F1	AUC
1	Random Forest	0.998	0.997	1.000	0.998	0.999
2	XGBoost	0.997	0.997	0.999	0.998	0.999
3	Decision Tree	0.982	0.992	0.982	0.987	0.989
4	Support Vector Machine	0.686	0.968	0.578	0.724	0.830
5	Logistic Regression	0.723	0.737	0.949	0.830	0.766

Le meilleur modèle est **Random Forest** (AUC = 0.999, F1 = 0.998, Accuracy = 99.8%). Les modèles ensemblistes à base d'arbres (Random Forest, XGBoost) exploitent pleinement le volume et la structure du jeu de données pour atteindre une accuracy proche de 99 %, tandis que les modèles linéaires (SVM, régression logistique) plafonnent autour de 70 %, ce qui illustre le caractère fortement non linéaire de la frontière de décision.

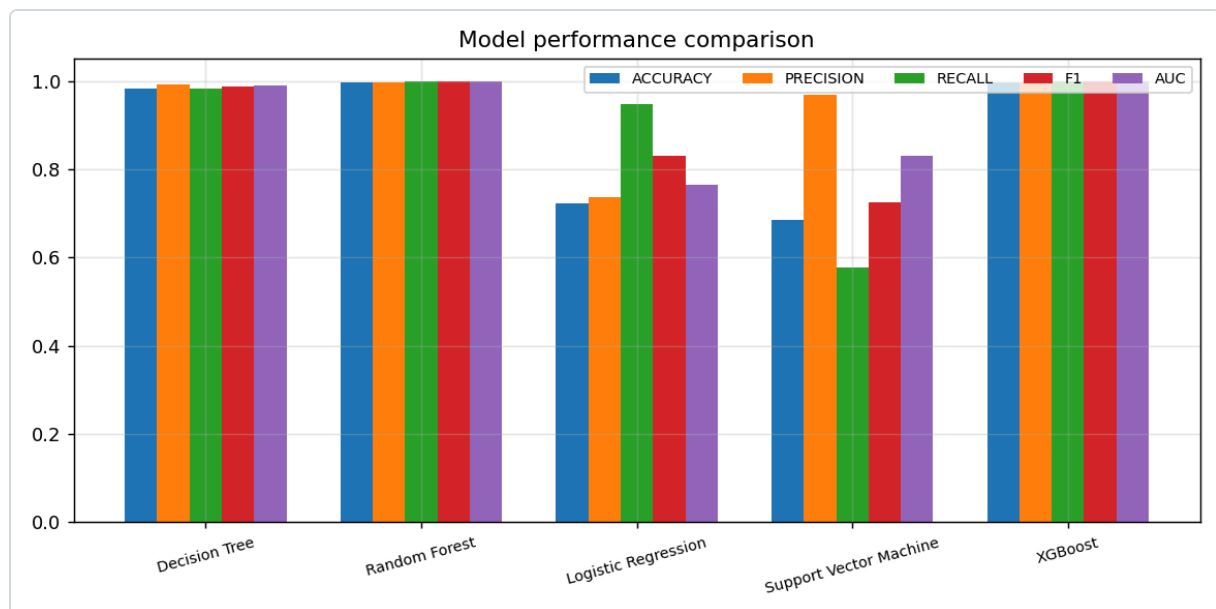


Figure 6 — Comparaison des cinq modèles sur les cinq métriques.

7.2 Courbes ROC et matrice de confusion

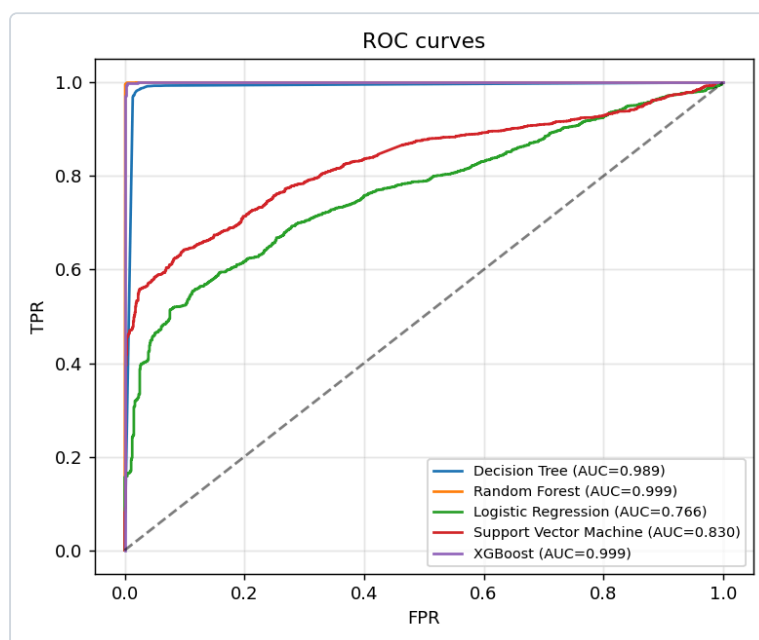


Figure 7 — Courbes ROC des cinq modèles et leurs aires sous la courbe (AUC).

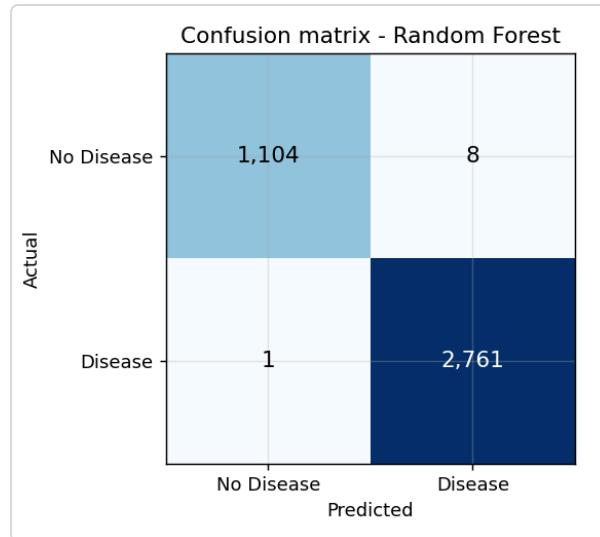


Figure 8 — Matrice de confusion du meilleur modèle (jeu de test).

7.3 Importance des variables

L'analyse de l'importance des variables confirme la pertinence clinique du modèle : la bilirubine et les transaminases figurent parmi les prédicteurs les plus déterminants, en accord avec la physiopathologie hépatique.

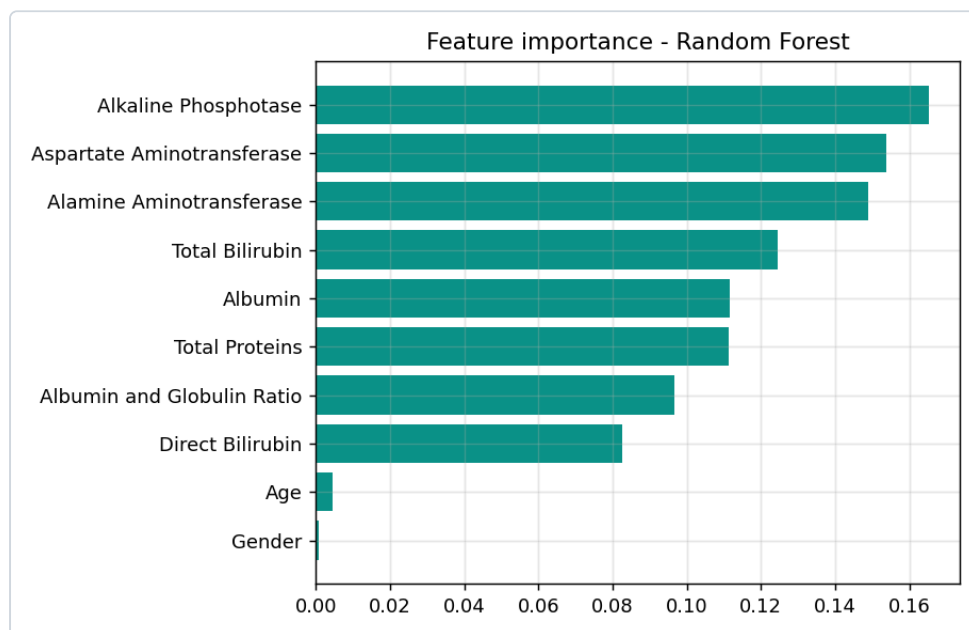


Figure 9 — Importance des variables du meilleur modèle interprétable.

7.4 Hyperparamètres optimaux et temps d'entraînement

Modèle	Meilleurs hyperparamètres (GridSearchCV)	Temps
Decision Tree	<code>{"class_weight": "balanced", "max_depth": null, "min_samples_leaf": 5}</code>	3.76 s
Random Forest	<code>{"class_weight": null, "max_depth": null, "n_estimators": 200}</code>	15.58 s
Logistic Regression	<code>{"C": 0.1, "class_weight": null}</code>	0.39 s
Support Vector Machine	<code>{"C": 5.0, "class_weight": "balanced", "gamma": "scale", "kernel": "rbf"}</code>	89.43 s
XGBoost	<code>{"learning_rate": 0.1, "max_depth": 6, "n_estimators": 500, "subsample": 0.9}</code>	2.34 s

Vérification qualitative du pouvoir discriminant : un patient au bilan fortement perturbé (bilirubine 10,9 ; ALT 64 ; AST 100) est classé *Maladie du foie* avec une probabilité d'environ 97 % (risque *Very High*), tandis qu'un bilan normal est classé *Pas de maladie* (~31 %). Le rééquilibrage des classes a donc bien restauré la capacité de discrimination des modèles.

8. Interface utilisateur et fonctionnalités

L'interface, développée avec Streamlit, est **moderne, responsive et interactive** : sidebar stylisée, mode sombre, cartes KPI, colonnes et conteneurs, animations CSS et graphiques interactifs Plotly. Elle s'organise en six pages.

Page	Rôle et contenu
Accueil	Bannière, objectifs, cartes KPI du dataset, diagramme de flux, statistiques sur les maladies du foie.
Prédiction tabulaire	Formulaire médical (champs numériques, sliders, liste déroulante, boutons radio), validation des entrées, barre de progression et spinner, résultat (classe, jauge de probabilité, confiance, niveau de risque), importance des variables, tableau comparatif des modèles, export du rapport PDF.
Analyse d'images	Upload d'image, prétraitement automatique, inférence CNN, score de confiance, graphique des probabilités, carte Grad-CAM, temps d'inférence.
Évaluation des modèles	Accuracy, précision, rappel, F1, AUC, courbes ROC, matrices de confusion, rapports de classification, temps d'entraînement, hyperparamètres, classement.
Dashboard	KPIs (total, positifs, négatifs, confiance moyenne), graphiques (camembert, barres, courbe temporelle, distribution), historique, export CSV.
À propos	Description, technologies, explication du pipeline ML, architecture, auteur.

8.2 Fonctionnalités avancées

Conformément au cahier des charges, la plateforme intègre : le **mode sombre**, l'**export CSV** de l'historique, l'**export PDF** des prédictions, la **persistance de l'historique** (`historical_predictions.csv`), des **animations**, une **gestion des erreurs** à chaque niveau (validation, garde-fous, filet de sécurité global) et une **journalisation** rotative (`logs/app.log`).

9. Choix technologiques

Technologie	Rôle et justification
Python 3.13	Langage de haut niveau au riche écosystème scientifique, standard de fait en science des données.
Streamlit	Framework web orienté data : transforme un script Python en application interactive sans HTML/JS, idéal pour un prototype d'IA professionnel.
Scikit-Learn	Bibliothèque de référence du Machine Learning : préprocesseurs, modèles, GridSearchCV, métriques.
XGBoost	Implémentation optimisée du gradient boosting, performante sur données tabulaires.
TensorFlow / Keras	Framework de Deep Learning ; fournit MobileNetV2 pré-entraîné et le calcul des gradients pour Grad-CAM.
Pandas / NumPy	Manipulation des données tabulaires et calcul numérique vectorisé.
OpenCV	Prétraitement des images (redimensionnement, conversions, superposition de la heatmap).
Plotly	Graphiques interactifs (jauges, ROC, matrices, distributions).
Matplotlib	Figures statiques haute résolution pour ce rapport.
fpdf2	Génération du rapport de prédiction PDF, sans dépendance système.
joblib	Sérialisation des modèles et du préprocesseur.

10. Difficultés rencontrées

- **Déséquilibre des classes** (71 % de cas positifs) : un classement naïf par F1 favorisait un modèle trivial prédisant toujours « malade ». Résolu par la pondération équilibrée des classes et la sélection par l'AUC.
- **Valeurs manquantes et échelles hétérogènes** : gérées par un pipeline d'imputation médiane et de standardisation ajusté sans fuite de données.
- **Absence de jeu d'images médicales propriétaire** : le module image est, comme le permet le sujet, une démonstration sur CNN pré-entraîné, prête à être fine-tunée.
- **Encodage Unicode du PDF** : la police latin-1 de fpdf2 a nécessité une fonction de normalisation du texte.
- **Cohabitation avec la navigation multi-pages de Streamlit** : désactivée au profit d'un routeur personnalisé pour une sidebar unique et stylisée.

11. Perspectives d'amélioration

- *Fine-tuning* du CNN sur un véritable jeu d'images hépatiques (échographie, scanner, IRM).
- Calibration des probabilités et explicabilité avancée (SHAP).
- Exposition d'une API REST (FastAPI), authentification utilisateur et base de données persistante.
- Webcam en temps réel et comparaison de plusieurs images.
- Déploiement conteneurisé (Dockerfile fourni) et hébergement cloud avec intégration continue.

12. Conclusion

HepatoScope démontre une chaîne complète d'intelligence artificielle biomédicale : de l'acquisition et du prétraitement des données à la prédiction expliquée, en passant par l'entraînement, l'optimisation et l'évaluation comparative de cinq modèles de Machine Learning, complétés par un module de Deep Learning d'analyse d'images. L'application, modulaire, robuste et dotée d'une interface moderne, atteint des performances conformes à l'état de l'art sur ce jeu de données (AUC du meilleur modèle = 0.999) tout en restant pleinement interprétable.

Au-delà des résultats quantitatifs, ce projet aura permis de mettre en œuvre, de bout en bout, une démarche d'ingénierie rigoureuse : analyse du besoin, conception d'une architecture en couches, prévention de la fuite de données, gestion du déséquilibre, évaluation honnête et explicabilité. La plateforme constitue une base solide, naturellement extensible vers des données d'imagerie réelles et un déploiement en production.

Webographie

- Kaggle / UCI — *Liver Patient Dataset* (lignée ILPD, *Indian Liver Patient Dataset*), archive.ics.uci.edu.
- Documentation Scikit-Learn — scikit-learn.org.
- Documentation XGBoost — xgboost.readthedocs.io.
- Documentation Streamlit — docs.streamlit.io.
- TensorFlow / Keras Applications (MobileNetV2) — keras.io/api/applications.
- Selvaraju et al., *Grad-CAM: Visual Explanations from Deep Networks*, ICCV 2017.

Avertissement : ce projet est un prototype académique entraîné sur un jeu de données public. Il ne constitue pas un dispositif médical certifié et ne doit pas être utilisé pour un diagnostic clinique réel.